1. Introduction to High Dimensional Data Analysis

Lieven Clement

Invalid Date

Table of contents

1 Introduction **2**
1.1 What are high dimensional data? . . . 2

2 Important tasks in high dimensional data analysis? **3**
2.1 Example: Kang et al. (2018)'s droplet-based scRNA-seq data of PBMCs cells from 8 lupus patients measured before and after 6h-treatment with INF-β (16 samples in total). . . . 3
2.2 Data exploration and dimensionality reduction . . . 4
2.3 Prediction . . . 6
2.4 Large scale hypothesis testing . . . 7

3 Linear regression **8**
3.1 Toy-Data . . . 8
3.2 Model . . . 9
3.2.1 Scalar form . . . 9
3.2.2 Scalar model for the toy dataset . . . 9
3.2.3 Vector/Matrix form . . . 10
3.2.4 Matrix form for toy dataset . . . 10
3.3 Interpretation . . . 10
3.4 Least Squares (LS) . . . 13
3.4.1 Minimize RSS . . . 13
3.4.2 Projection . . . 14
3.4.3 Error . . . 34
3.4.4 Curse of dimensionality? . . . 34
3.5 Variance Estimator? . . . 41
3.6 Prediction error . . . 41

1 Introduction

- We live in a big data era
- Massive Datasets on our location, surfing behavior, consumer data, social media, …
- Life Sciences: advent of high throughput technologies has enabled us to measure brain activity (brain images) as well as the expression of thousands of genes, proteins, … for each subject or even single cell.
- Industry: process control with many sensors that follow process in real time…
- Data drive journalism
- …

Challenge: We have to learn from high dimensional data!

1.1 What are high dimensional data?

- We typically observe multiple variables/features (p) for each subject/experimental unit $i = 1, \dots, n$ i.e.

$$\mathbf{x}_i^T = [x_{i1}, \dots, x_{ip}]$$

- Multivariate statistics have a long history, but were designed for the $n >>> p$ case,
- Nowadays many high throughput technologies generate multivariate data with many variables (large p) as compared to the number of independent replicates or samples (sample size n), resulting in **{high-dimensional data}**, which is characterised by

$$p >>> n.$$

- New statistical methods for dealing with the $p >>> n$ case have been developed in the last 20 years. Some of them are adaptations of multivariate methods.

Issues with high-dimensional data:

- *Computational problems*: large matrices, numerical accuracy of computers become an issue

- *Classical asymptotic theory does not hold* for $p \to \infty$ as $n \to \infty$
- *Model (or feature) selection* requires specialised methods to deal with the enormous number of possible models. (e.g. in linear regression with p potential predictors: 2^p possible models)
- Models that can potentially depend on large-p predictors are vulnerable to *huge overfitting*.
- In searching for associations between an outcome and large p potential exploratory variables, we are at risk to make *many false discoveries*
- The *Curse of Dimensionality* may cause a prediction model to become useless. (n data points become sparse in large-p dimensional space)

---

2 Important tasks in high dimensional data analysis?

2.1 Example: Kang et al. (2018)'s droplet-based scRNA-seq data of PBMCs cells from 8 lupus patients measured before and after 6h-treatment with INF-β (16 samples in total).

```
library(tidyverse)

## Packages to load and visualize the single-cell data
library(ExperimentHub)
library(scater)
```

```
## Load Kang single-cell data from ExperimentHub
## Use the local cache if the data were downloaded before (avoids a slow
## online metadata check); otherwise connect to the hub and download once
eh <- tryCatch(ExperimentHub(localHub = TRUE), error = function(e) NULL)
if (is.null(eh) || !"EH2259" %in% names(eh)) eh <- ExperimentHub()
sce <- eh[["EH2259"]]

sce
```

```
#> class: SingleCellExperiment
#> dim: 35635 29065
#> metadata(0):
```

```
#> assays(1): counts
#> rownames(35635): MIR1302-10 FAM138A ... MT-ND6 MT-CYB
#> rowData names(2): ENSEMBL SYMBOL
#> colnames(29065): AAACATACAATGCC-1 AAACATACATTTCC-1 ... TTTGCATGGTTTGG-1
#>   TTTGCATGTCTTAC-1
#> colData names(5): ind stim cluster cell multiplets
#> reducedDimNames(1): TSNE
#> mainExpName: NULL
#> altExpNames(0):
```

- Data on gene expression of 35635 genes of 29065 cells.

2.2 Data exploration and dimensionality reduction

- Visualisation is a first essential step to learn from data

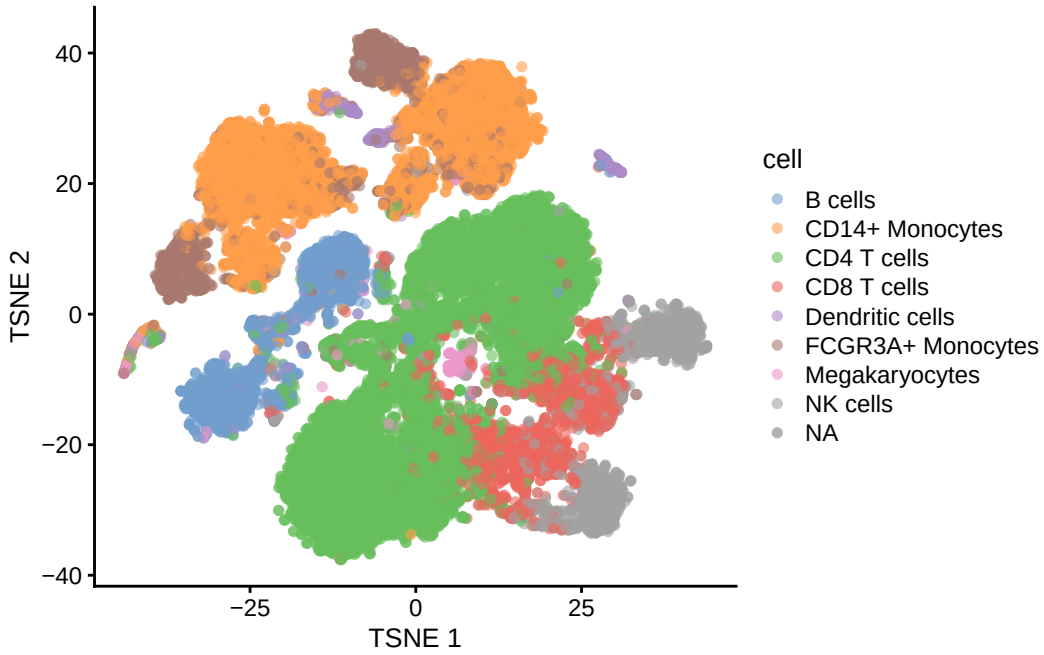
```
counts(sce)[18990:19000,9:20]
```

```
#> 11 x 12 sparse Matrix of class "dgCMatrix"
```

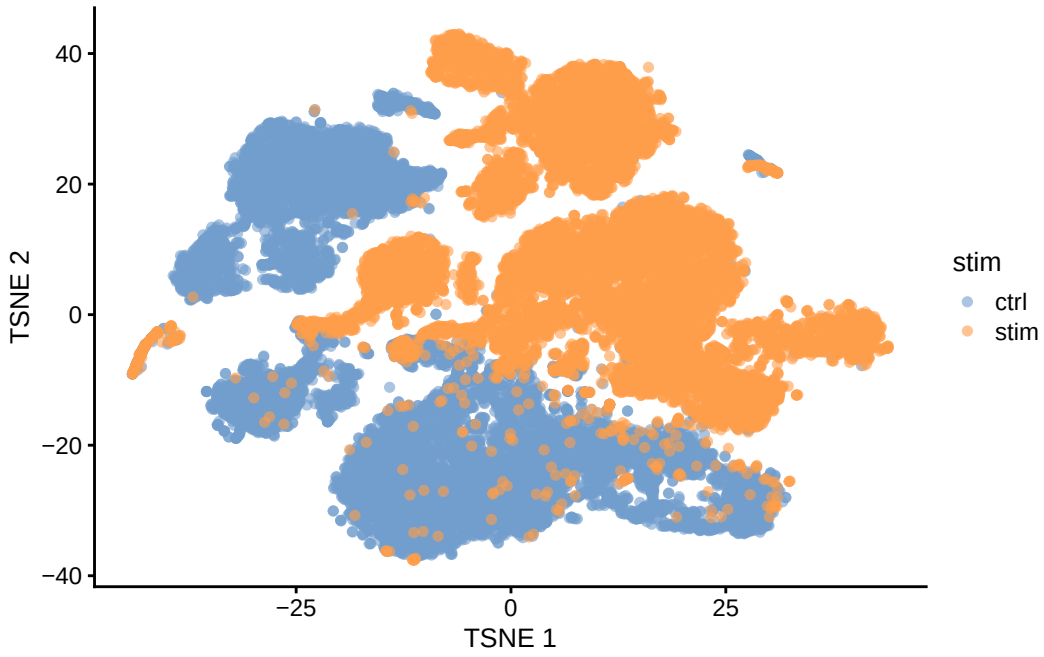
```
#>
#> RP11-467L20.10   . .   . . .   .  .  .  . .   .   .
#> MYRF             . .   . . .   .  .  .  . .   .   .
#> TMEM258          . .   3 . .   .  1  1  . .   .   .
#> FEN1             . .   . . .   .  .  .  . .   .   .
#> FADS2            . .   . . .   .  .  .  . .   .   .
#> FADS1            . .   . . .   .  .  .  . .   .   .
#> FADS3            . .   . . .   .  .  .  . .   .   .
#> RAB3IL1          . .   . . .   .  .  .  . .   .   .
#> BEST1            1 .   . . .   .  1  .  . .   1   1
#> FTH1           163 8 271 4 5 174 38 58 31 3 663 612
#> AP003733.1       . .   . . .   .  .  .  . .   .   .
```

- It is impossible to learn about the structure in the data by staring at the data matrix.
- We should be able to explore the data in a low dimensional projection

```
plotReducedDim(sce, dimred="TSNE", colour_by="cell")
```




```
plotReducedDim(sce, dimred="TSNE", colour_by="stim")
```

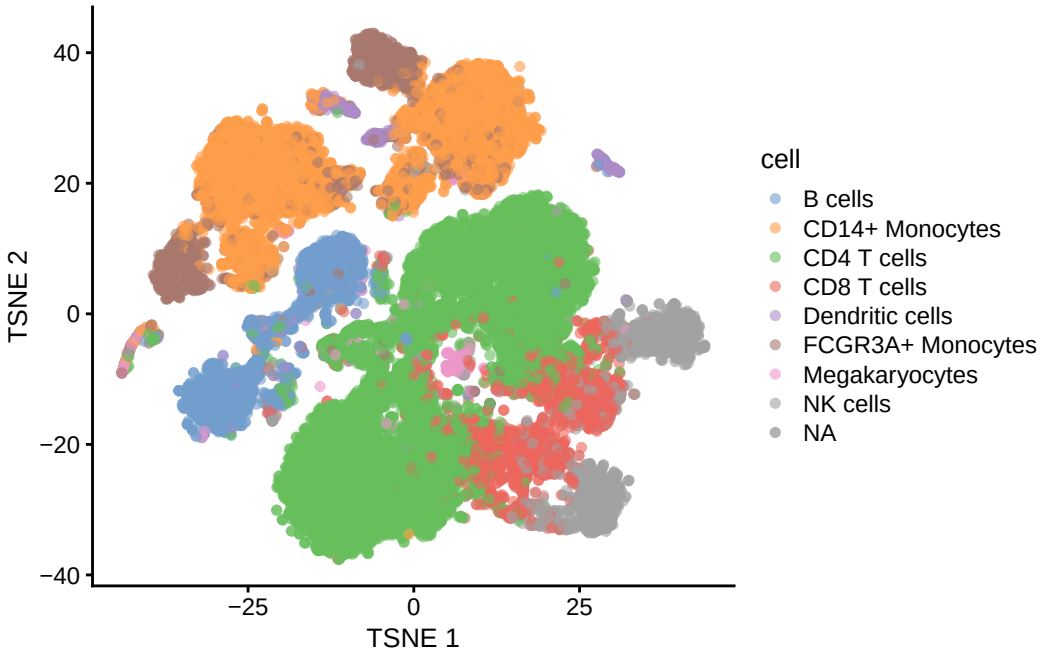


- Note, that we see huge effect of treatment. If I see this I am always on my guard!

- We contacted the authors and learned that all control cells were sequenced in a first run and all stimulated cells were on a second sequencing run. So the large effect might be an effect of batch!

2.3 Prediction

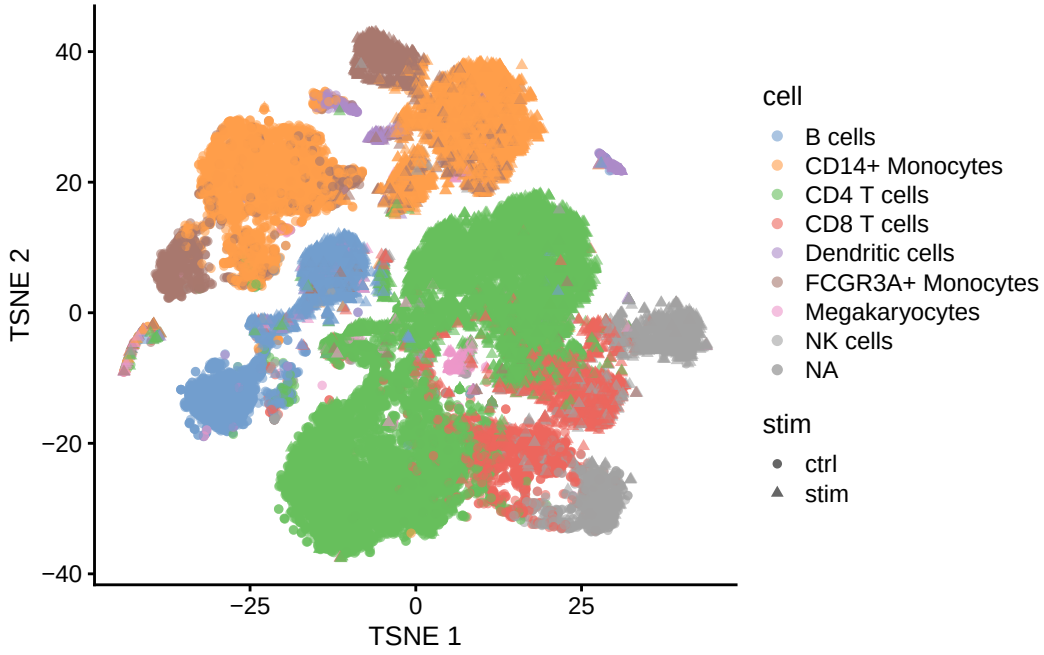
```
plotReducedDim(sce, dimred="TSNE", colour_by="cell")
```



- In single cell analysis it is key to identify cell types in scRNA-seq data sets before in-depth investigations of their functional and pathological roles.
- Use models that were build based on reference data sets to predict cell types in new data sets based on their gene expression pattern
- Problem: we have 35635 genes at our disposal to build this prediction model!
- Other examples
  - Prediction of risk on mortality is used on a daily basis in intensive care units to prioritise patient care.
  - Facebook: predict the identity of the faces of people that are on an new image that is uploaded.
  - Netflix: Suggest movies that you would like

2.4 Large scale hypothesis testing

```
plotReducedDim(sce, dimred="TSNE", colour_by="cell", shape_by="stim")
```



- Which genes are differentially expressed between control and stimulated treatment (assess in each cell type)
- For which genes we see that the differential expression according to treatment is changing according to the cell type (interaction)
- We have to model the gene expression for each gene

$$\begin{cases} y_{ig} & \sim & NB(\mu_{ig}, \phi_g) \\ E[y_{ig}] & = & \mu_{ig} \\ \log(\mu_{ig}) & = & \eta_{ig} \\ \eta_{ig} & = & \beta_{0,g} + \sum_{c=2}^{C} \beta_{c,g} X_{ic} + \beta_{s,g} X_{is} + \sum_{c=2}^{C} \beta_{c:s,g} X_{ic} X_{is} + \sum_{e=2}^{E} \beta_{e,g} X_{ie} + \alpha_{ig} \end{cases}$$

With

- Cell type $c = 2 \dots C$ and X_{ic} is a dummy variable that $X_{ic} = 1$ if cell i is of cell type c and $X_{ic} = 0$ otherwise. Note that cell type $c = 1$ is the reference cell type

- Indicator X_{is} indicates if cell i was stimulated $X_{is} = 1$ or not $X_{is} = 0$. So the control treatment is the reference treatment.
- Experimental unit $e = 2 \dots E$ and X_{ie} a dummy variable that $X_{ie} = 1$ if cell i originates of experimental unit (subject) e and $X_{ie} = 0$ otherwise. Note that cell type $e = 1$ is the reference experimental unit.
- α_{ig} a cell specific and gene specific normalisation factor

Suppose we want to test if the effect of the stimulus (average difference in expression between stimulated and non stimulated cells) is different in cell type c than in the reference cell type 1?

- $H_0 : \beta_{c:s,g} = 0$
- $H_1 : \beta_{c:s,g} \neq 0$
- We have to assess this for 35635 genes!
- If we assess each test at the 5% level we can expect 0.05 * 35635 = 1782 false positives.

$\rightarrow$ massive multiple testing problem!

Note, that we cannot differentiate between batch and treatment because of the flaw in the experimental design!

Other examples

- Find regions (voxels) in the brain (on brain image) that are associated with a certain condition/treatment
- Evaluation of trading rules

---

3 Linear regression

- Linear regression is a very important statistical tool to study the association between variables and to build prediction models.

3.1 Toy-Data

Consider the following toy-dataset with 3 observation (X,Y):

```r
library(tidyverse)
data <- data.frame(x=1:3,y=c(1,2,2))
data
```

```
#>   x y
#> 1 1 1
#> 2 2 2
#> 3 3 2
```

3.2 Model

3.2.1 Scalar form

- Consider a vector of predictors $\mathbf{x} = (x_1, \dots, x_p)$ and
- a real-valued response Y
- then the linear regression model can be written as

$$Y = f(\mathbf{x}) + \epsilon = \beta_0 + \sum_{j=1}^{p} x_j \beta_j + \epsilon$$

  with i.i.d. $\epsilon \sim N(0, \sigma^2)$
- We will often work on mean centered variables: $Y_c = Y - \bar{Y}$ and $X_c = X - \bar{X}$, then the intercept is dropped from the model:

$$Y_c = f(\mathbf{x}_c) + \epsilon = \sum_{j=1}^{p} x_{cj} \beta_j + \epsilon$$

3.2.2 Scalar model for the toy dataset

$$y_i = \beta_0 + \beta_1 x + \epsilon_i$$

If we write the model for each observation:

$$\begin{aligned} 1 &= \beta_0 + \beta_1 1 + \epsilon_1 \\ 2 &= \beta_0 + \beta_1 2 + \epsilon_2 \\ 2 &= \beta_0 + \beta_1 3 + \epsilon_3 \end{aligned}$$

3.2.3 Vector/Matrix form

- n observations $(\mathbf{x}_1, y_1) \ldots (\mathbf{x}_n, y_n)$ with $\mathbf{x}_1^T = [\ 1 \quad x_1 \quad \ldots \quad x_p\ ]$
- Regression in matrix notation

$$\mathbf{Y} = \mathbf{X}\ +$$

with $\mathbf{Y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$, $\mathbf{X} = \begin{bmatrix} 1 & x_{11} & \ldots & x_{1p} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \ldots & x_{np} \end{bmatrix}$ or $\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_n^T \end{bmatrix}$, $\beta = \begin{bmatrix} \beta_0 \\ \vdots \\ \beta_p \end{bmatrix}$ and $= \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}$

Note, that upon centering of $\mathbf{X}$ and $\mathbf{Y}$ the 1 is dropped from each $\mathbf{x}_i$ and thus the column of 1 is dropped in $\mathbf{X}_c$

3.2.4 Matrix form for toy dataset

We can also write this in matrix form

$$\mathbf{Y} = \mathbf{X}\beta + \epsilon$$

with

$$\mathbf{Y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} \quad \text{and} \quad \epsilon = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \end{bmatrix}$$

3.3 Interpretation

From the linear regression we get

$$E[Y \mid \mathbf{x}] = \mathbf{x}^T\beta.$$

- Hence, the β parameters relate the predictor $\mathbf{x}$ to the mean outcome.
- If we know the covariate pattern $\mathbf{x}$ we can use the model to predict Y.

For a model with a single predictor we obtain

$$E[Y \mid x] = \beta_0 + \beta_1 x$$

and

$$\beta_1 = E[Y \mid x+1] - E[Y \mid x] = \beta_0 + \beta_1(x+1) - \beta_0 - \beta_1 x.$$

In this course we will typically center Y and X and then we get the following

$$E[Y_c \mid x_c] = \beta_1 x_c$$

and

$$\beta_1 = E[Y_c \mid x_c + 1] - E[Y_c \mid x_c].$$

Note, that the estimator for β_1 will be exactly same when estimated based on the models with/or without centering.

- The parameter β_1 has an interpretation as the average difference in the outcome between subjects that differ with one unit for the predictor.
- The parameter β_1 does not say much about individual outcomes. The residual variance σ^2 determines how much individual outcomes vary about the mean outcome.

The β parameters are used to measure association, but a $\beta \neq 0$ does not necessarily mean that the model will give good predictions.

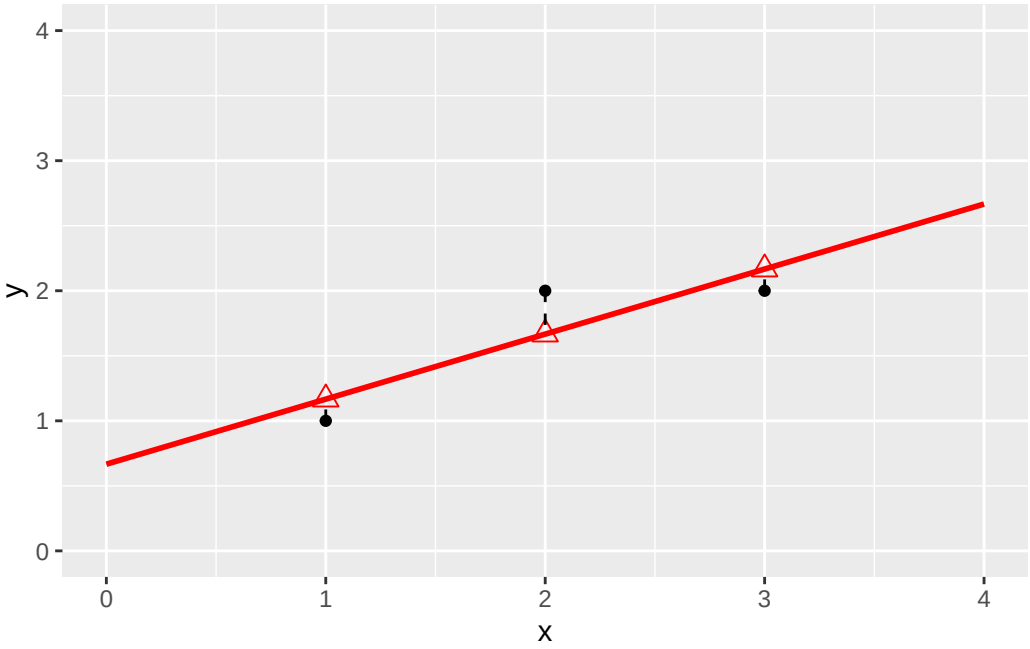
$$\hat{Y} = \mathbf{x}^T \hat{\beta}$$

- In Chapter 3 will we will discuss the prediction problem for high dimensional data

Model fit and predictions based on the toy dataset

```
lm1 <- lm(y~x,data)
data$yhat <- lm1$fitted

data |>
  ggplot(aes(x,y)) +
  geom_point() +
  ylim(0,4) +
  xlim(0,4) +
  stat_smooth(method = "lm", color = "red", fullrange = TRUE) +
  geom_point(aes(x=x, y =yhat), pch = 2, size = 3, color = "red") +
  geom_segment(data = data, aes(x = x, xend = x, y = y, yend = yhat), lty = 2 )
```



- In a Chapter 6 we will discuss the problem of large scale hypothesis testing: testing many hypotheses in a single study (ten to hundred thousands of hypotheses).
  - A statistical test is constructed to control the type I error rate at the significance level α to assess the null hypothesis that there is no association between a predictor and the outcome vs the alternative hypothesis that there is an association between a predictor and the outcome.

$$H_0 : \beta_1 = 0 \text{ vs } H_1 : \beta_1 \neq 0.$$

  - However, when many hypotheses are to be tested in a single study, the probability to find false associations is no longer controlled if p-values are compared to the significance level α.
  - We will later introduce the concept of false discovery rates to overcome the problem.

3.4 Least Squares (LS)

- Minimize the residual sum of squares

$$\begin{aligned} RSS(\beta) &= \sum_{i=1}^{n} e_i^2 \\ &= \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 \end{aligned}$$

- or in matrix notation

$$\begin{aligned} RSS(\beta) &= \mathbf{e}^T\mathbf{e} \\ &= \begin{bmatrix} e_1 & \dots & e_n \end{bmatrix} \begin{bmatrix} e_1 \\ \vdots \\ e_n \end{bmatrix} \\ &= e_1^2 + e_2^2 + \dots + e_n^2 \\ &= (\mathbf{Y} - \mathbf{X})^T(\mathbf{Y} - \mathbf{X}) \\ &= \|\mathbf{Y} - \mathbf{X}\|_2^2 \end{aligned}$$

with the L_2-norm of a p-dim. vector v $\|\mathbf{v}\|_2 = \sqrt{v_1^2 + \dots + v_p^2} \rightarrow \widehat{\beta} = \text{argmin}_\beta \|\mathbf{Y} - \mathbf{X}\|_2^2$

3.4.1 Minimize RSS

$$\begin{aligned} \frac{\partial RSS}{\partial \beta} &= \mathbf{0} \\ \frac{(\mathbf{Y}-\mathbf{X})^T(\mathbf{Y}-\mathbf{X}\beta)}{\partial \beta} &= \mathbf{0} \\ -2\mathbf{X}^T(\mathbf{Y} - \mathbf{X}\beta) &= \mathbf{0} \\ \mathbf{X}^T\mathbf{X} &= \mathbf{X}^T\mathbf{Y} \\ \widehat{\beta} &= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y} \end{aligned}$$

It can be shown that the estimator is unbiased:

$$E[\hat{\beta}] = \beta$$

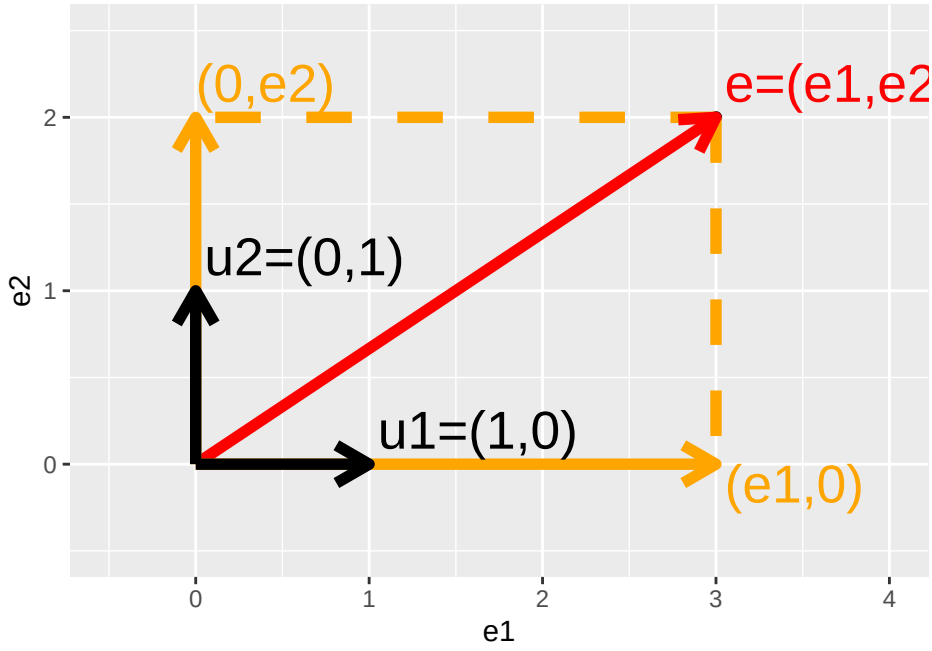
3.4.2 Projection

There is also another picture to regression:

- Instead of plotting each observation $i = 1 \dots n$ as a data-point in $\mathbb{R}^p$ with dimensions $1 \dots p$ for every variable/feature that is recorded for each observation
- We can also plot $\mathbf{Y}$, $\hat{\mathbf{Y}}$ and each column of $\mathbf{X}$: $\mathbf{X}_j$ with $j = 1 \dots p$ as a vector in $\mathbb{R}^n$ with dimensions $1 \dots n$ for every observation.
- In this representation linear regression can be interpreted as a projection of the vector $\mathbf{Y}$ onto the subspace of $\mathbb{R}^n$ that is spanned by the vectors for the predictors $\mathbf{X}_1 \dots \mathbf{X}_p$.
- The space $\mathbf{X}_1 \dots \mathbf{X}_p$ is also referred to as the column space of $\mathbf{X}$, the space that consists of all linear combinations of the vectors of the predictors or columns $\mathbf{X}_1 \dots \mathbf{X}_p$.

3.4.2.1 Intermezzo: Projection of vector on X and Y axis

$$\mathbf{e} = \begin{bmatrix} e_1 \\ e_2 \end{bmatrix}, \mathbf{u}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \mathbf{u}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$



1. Projection of error on x-axis

$$\begin{aligned} \mathbf{u}_1^T\mathbf{e} &= \|\mathbf{u}_1\|_2\|\mathbf{e}_1\|_2 \cos < \mathbf{u}_1, \mathbf{e}_1 > \\ &= \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} \\ &= 1 \times e_1 + 0 \times e_2 \\ &= e_1 \end{aligned}$$

2. Projection of error on y-axis

$$\begin{aligned} \mathbf{u}_2^T\mathbf{e} &= \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} \\ &= 0 \times e_1 + 1 \times e_2 \\ &= e_2 \end{aligned}$$

3. Projection of error on itself

$$
\begin{aligned}
\mathbf{e}^T\mathbf{e} &= \begin{bmatrix} e_1 & e_2 \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} \\
&= e_1^2 + e_2^2 \\
&= \|e\|_2^2 \rightarrow \text{ Pythagorean theorem}
\end{aligned}
$$

3.4.2.2 Interpretation of least squares as a projection

Fitted values:

$$
\begin{aligned}
\hat{\mathbf{Y}} &= \mathbf{X}\hat{\beta} \\
&= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y} \\
&= \mathbf{H}\mathbf{Y}
\end{aligned}
$$

with $\mathbf{H}$ the projection matrix also referred to as the hat matrix.

```
X <- model.matrix(~x,data)
X
```

```
#>   (Intercept) x
#> 1           1 1
#> 2           1 2
#> 3           1 3
#> attr(,"assign")
#> [1] 0 1
```

```
XtX <- t(X)%*%X
XtX
```

```
#>             (Intercept)  x
#> (Intercept)           3  6
#> x                     6 14
```

```
XtXinv <- solve(t(X)%*%X)
XtXinv
```

```
#>             (Intercept)    x
#> (Intercept)    2.333333 -1.0
#> x             -1.000000  0.5
```

```
H <- X %*% XtXinv %*% t(X)
H
```

```
#>           1         2          3
#> 1  0.8333333 0.3333333 -0.1666667
#> 2  0.3333333 0.3333333  0.3333333
#> 3 -0.1666667 0.3333333  0.8333333
```

```
Y <- data$y
Yhat <- H%*%Y
Yhat
```

```
#>       [,1]
#> 1 1.166667
#> 2 1.666667
#> 3 2.166667
```

- We can also interpret the fit as the projection of the $n \times 1$ vector $\mathbf{Y}$ on the column space of the matrix $\mathbf{X}$.
- So each column in $\mathbf{X}$ is also an $n \times 1$ vector.
- For the toy example n=3 and p=2. The other picture to linear regression is to consider X_0, X_1 and Y as vectors in the space of the data $\mathbb{R}^n$, here $\mathbb{R}^3$ because we have three data points. So the column space of X is a plane in the three dimensional space.

$$\hat{\mathbf{Y}} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

1. Plane spanned by column space: The other picture to linear regression is to consider X_0, X_1 and Y as vectors in the space of the data $\mathbb{R}^n$, here $\mathbb{R}^3$ because we have three data points.

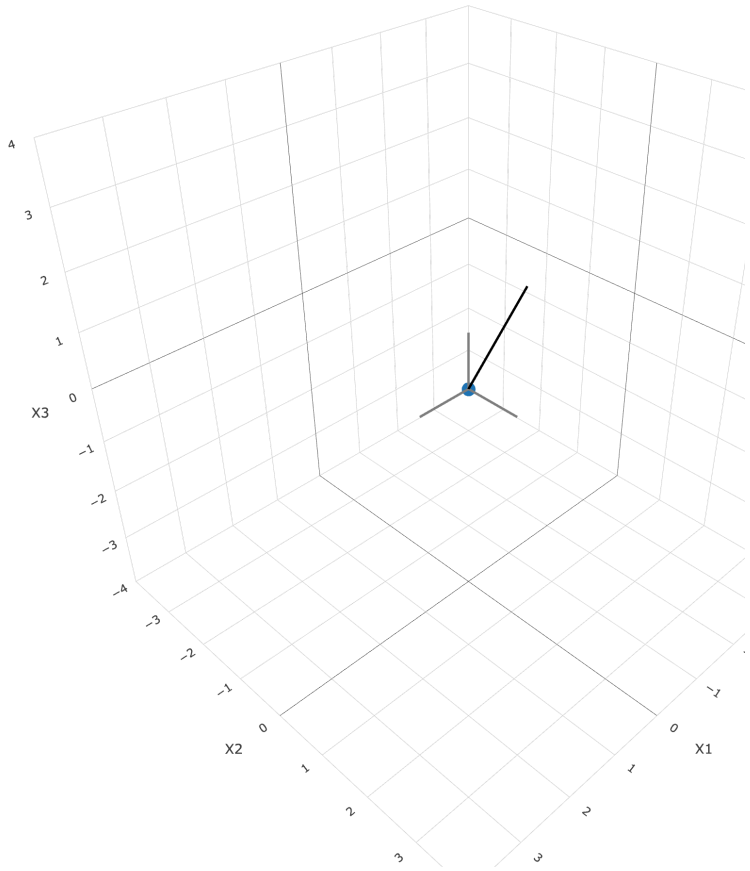
```
originRn <- data.frame(X1=0,X2=0,X3=0)
data$x0 <- 1
dataRn <- data.frame(t(data))

library(plotly)

p1 <- plot_ly(
    originRn,
    x = ~ X1,
```

```
    y = ~ X2,
    z= ~ X3, name="origin") |>
  add_markers(type="scatter3d") |>
  layout(
    scene = list(
      aspectmode="cube",
      xaxis = list(range=c(-4,4)), yaxis = list(range=c(-4,4)), zaxis = list(range=c(-4,4))
      )
    )
p1 <- p1 |>
  add_trace(
    x = c(0,1),
    y = c(0,0),
    z = c(0,0),
    mode = "lines",
    line = list(width = 5, color = "grey"),
    type="scatter3d",
    name = "obs1") |>
  add_trace(
    x = c(0,0),
    y = c(0,1),
    z = c(0,0),
    mode = "lines",
    line = list(width = 5, color = "grey"),
    type="scatter3d",
    name = "obs2") |>
  add_trace(
    x = c(0,0),
    y = c(0,0),
    z = c(0,1),
    mode = "lines",
    line = list(width = 5, color = "grey"),
    type="scatter3d",
    name = "obs3") |>
  add_trace(
    x = c(0,1),
    y = c(0,1),
    z = c(0,1),
    mode = "lines",
    line = list(width = 5, color = "black"),
    type="scatter3d",
    name = "X1") |>
```

```
    add_trace(
    x = c(0,1),
    y = c(0,2),
    z = c(0,3),
    mode = "lines",
    line = list(width = 5, color = "black"),
    type="scatter3d",
    name = "X2")
p1
```


X3
X2
X1

2. Vector of Y:

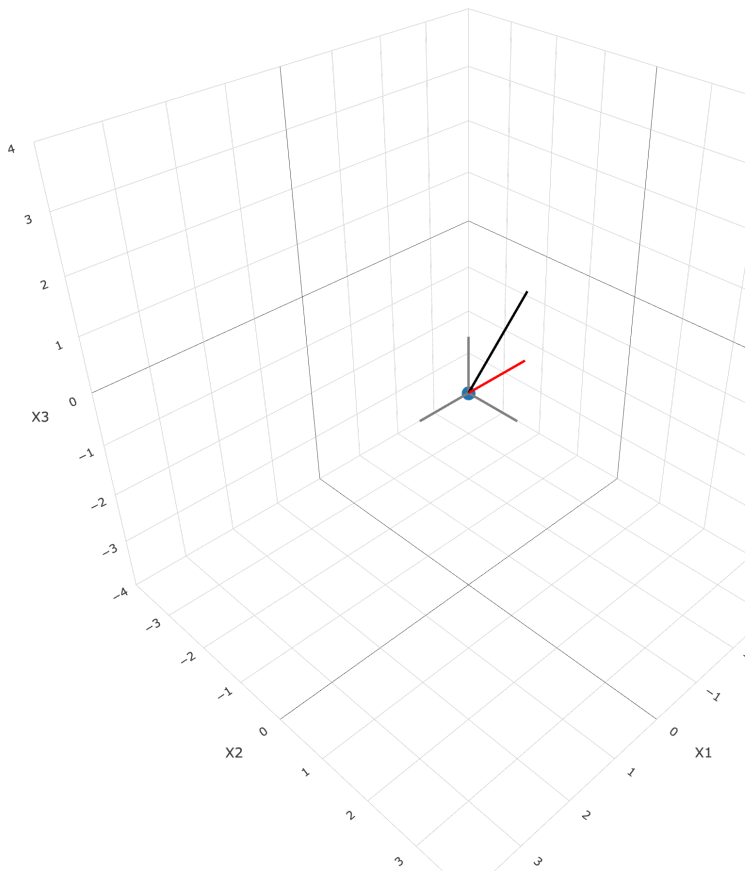
Actual values of $\mathbf{Y}$:

```
data$y
```

```
#> [1] 1 2 2
```

$$\mathbf{Y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$$

```
p2 <- p1 |>
  add_trace(
    x = c(0,Y[1]),
    y = c(0,Y[2]),
    z = c(0,Y[3]),
    mode = "lines",
    line = list(width = 5, color = "red"),
    type="scatter3d",
    name = "Y")
p2
```


4
3
2
1
0
−1
−2
−3
−4
X3
−3
−2
−1
0
1
2
3
X2
−1
0
1
2
3
X1

3. Projection of Y onto column space

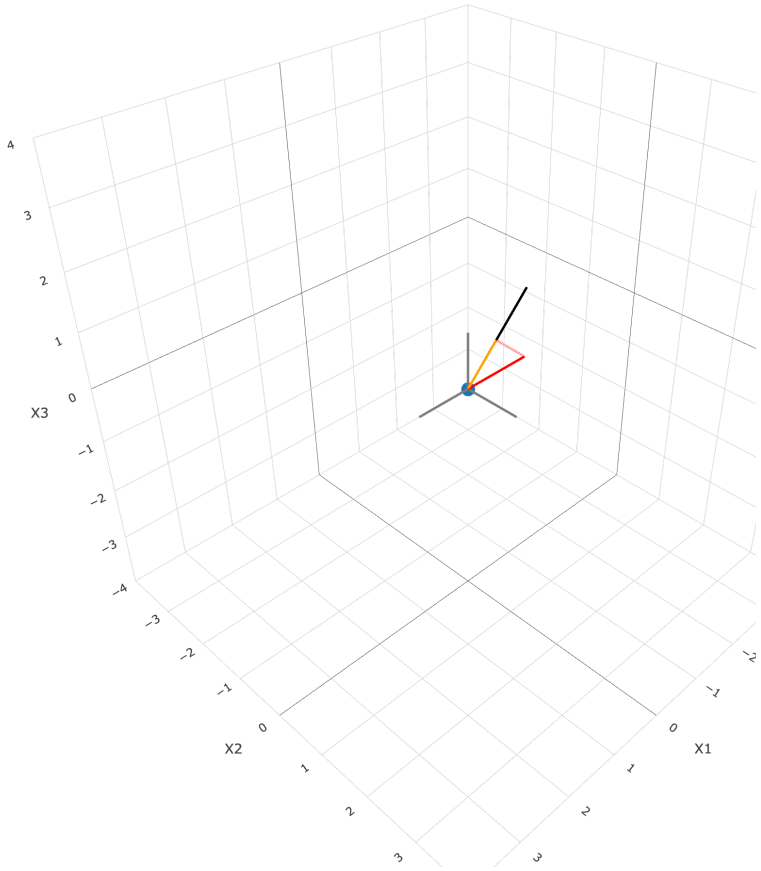
Actual values of fitted values $\hat{\mathbf{Y}}$:

```
data$yhat
```

```
#> [1] 1.166667 1.666667 2.166667
```

$$\mathbf{Y} = \begin{bmatrix} 1.1666667 \\ 1.6666667 \\ 2.1666667 \end{bmatrix}$$

```
p2 <- p2 |>
  add_trace(
    x = c(0,Yhat[1]),
    y = c(0,Yhat[2]),
    z = c(0,Yhat[3]),
    mode = "lines",
    line = list(width = 5, color = "orange"),
    type="scatter3d",
    name="Yhat") |>
    add_trace(
    x = c(Y[1],Yhat[1]),
    y = c(Y[2],Yhat[2]),
    z = c(Y[3],Yhat[3]),
    mode = "lines",
    line = list(width = 5, color = "red", dash="dash"),
    type="scatter3d",
    name="Y -> Yhat"
    )
p2
```


X3
X2
X1

$\mathbf{Y}$ is projected in the column space of $\mathbf{X}$! spanned by the columns.

3.4.2.3 How does this projection works?

$$\begin{aligned}\hat{\mathbf{Y}} &= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y} \\ &= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1/2}(\mathbf{X}^T\mathbf{X})^{-1/2}\mathbf{X}^T\mathbf{Y} \\ &= \mathbf{U}\mathbf{U}^T\mathbf{Y}\end{aligned}$$

- $\mathbf{U}$ is a new orthonormal basis in $\mathbb{R}^2$, a subspace of $\mathbb{R}^3$
- The space spanned by U and X is the column space of X, e.g. it contains all possible linear combinations of X. $\mathbf{U}^t\mathbf{Y}$ is the projection of Y on this new orthonormal basis

```
eigenXtX <- eigen(XtX)
XtXinvSqrt <- eigenXtX$vectors %*%diag(1/eigenXtX$values^.5)%*%t(eigenXtX$vectors)
U <- X %*% XtXinvSqrt
```

- $\mathbf{U}$ orthonormal basis

```
U
```

```
#>          [,1]        [,2]
#> 1  0.9116067 -0.04802616
#> 2  0.3881706  0.42738380
#> 3 -0.1352655  0.90279376
```

```
t(U)%*%U
```

```
#>               [,1]         [,2]
#> [1,] 1.000000e+00 2.915205e-16
#> [2,] 2.915205e-16 1.000000e+00
```

- $\mathbf{U}\mathbf{U}^T$ equals projection matrix

```
U%*%t(U)
```

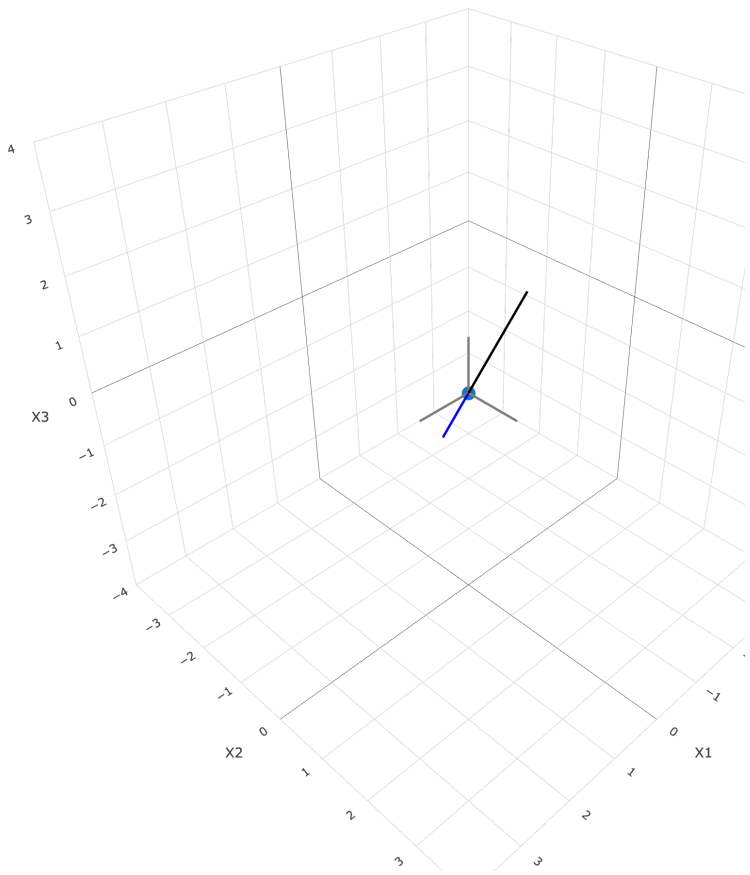
```
#>            1         2          3
#> 1  0.8333333 0.3333333 -0.1666667
#> 2  0.3333333 0.3333333  0.3333333
#> 3 -0.1666667 0.3333333  0.8333333
```

```
H
```

```
#>            1         2          3
#> 1  0.8333333 0.3333333 -0.1666667
#> 2  0.3333333 0.3333333  0.3333333
#> 3 -0.1666667 0.3333333  0.8333333
```

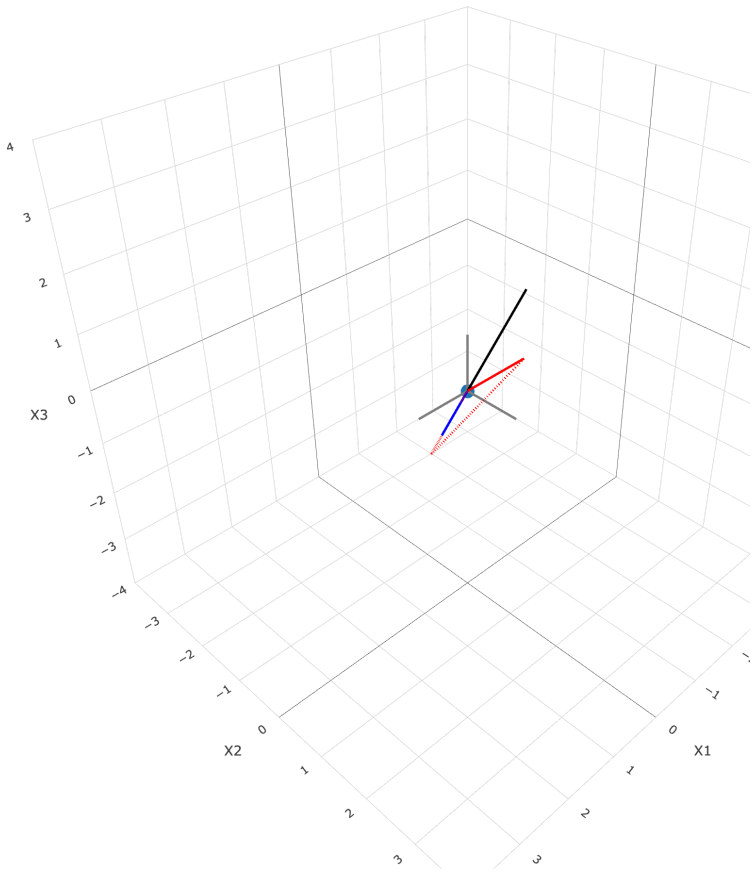
```
p3 <- p1 |>
  add_trace(
    x = c(0,U[1,1]),
    y = c(0,U[2,1]),
    z = c(0,U[3,1]),
    mode = "lines",
    line = list(width = 5, color = "blue"),
    type="scatter3d",
    name = "U1") |>
  add_trace(
    x = c(0,U[1,2]),
    y = c(0,U[2,2]),
    z = c(0,U[3,2]),
    mode = "lines",
    line = list(width = 5, color = "blue"),
    type="scatter3d",
    name = "U2")

p3
```


X3
X2
X1

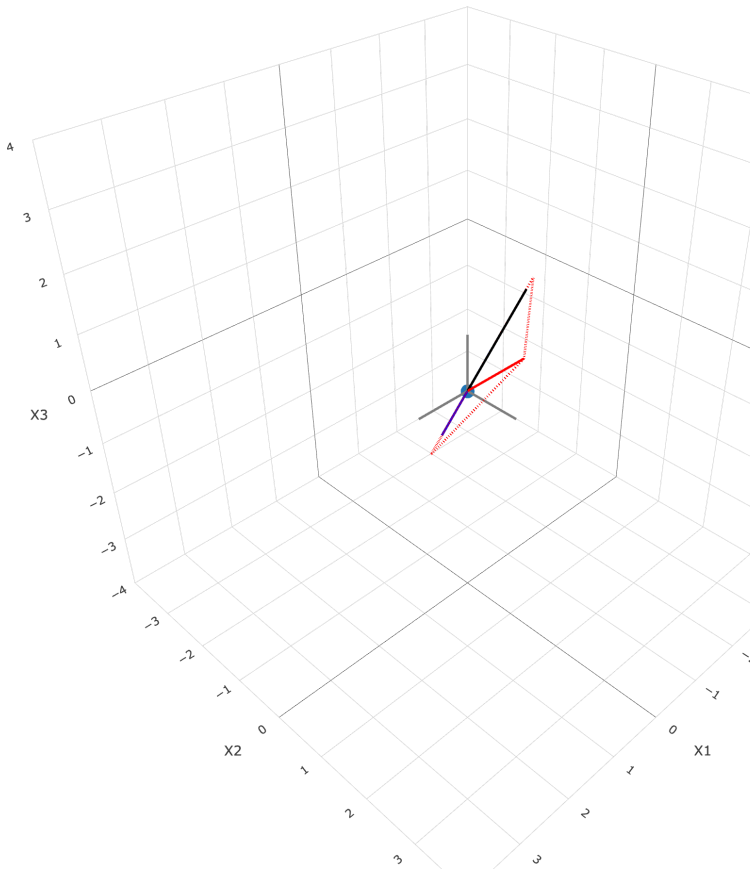
- $\mathbf{U}^T\mathbf{Y}$ is the projection of $\mathbf{Y}$ in the space spanned by $\mathbf{U}$.
- Indeed $\mathbf{U}_1^T\mathbf{Y}$

```
p4 <- p3 |>
  add_trace(
    x = c(0,Y[1]),
    y = c(0,Y[2]),
    z = c(0,Y[3]),
    mode = "lines",
    line = list(width = 5, color = "red"),
    type="scatter3d",
    name = "Y") |>
  add_trace(
    x = c(0,U[1,1]*(U[,1]%*%Y)),
    y = c(0,U[2,1]*(U[,1]%*%Y)),
    z = c(0,U[3,1]*(U[,1]%*%Y)),
    mode = "lines",
    line = list(width = 5, color = "red",dash="dash"),
    type="scatter3d",
    name="Y -> U1") |> add_trace(
    x = c(Y[1],U[1,1]*(U[,1]%*%Y)),
    y = c(Y[2],U[2,1]*(U[,1]%*%Y)),
    z = c(Y[3],U[3,1]*(U[,1]%*%Y)),
    mode = "lines",
    line = list(width = 5, color = "red", dash="dash"),
    type="scatter3d",
    name="Y -> U1")
p4
```


X3
X2
X1

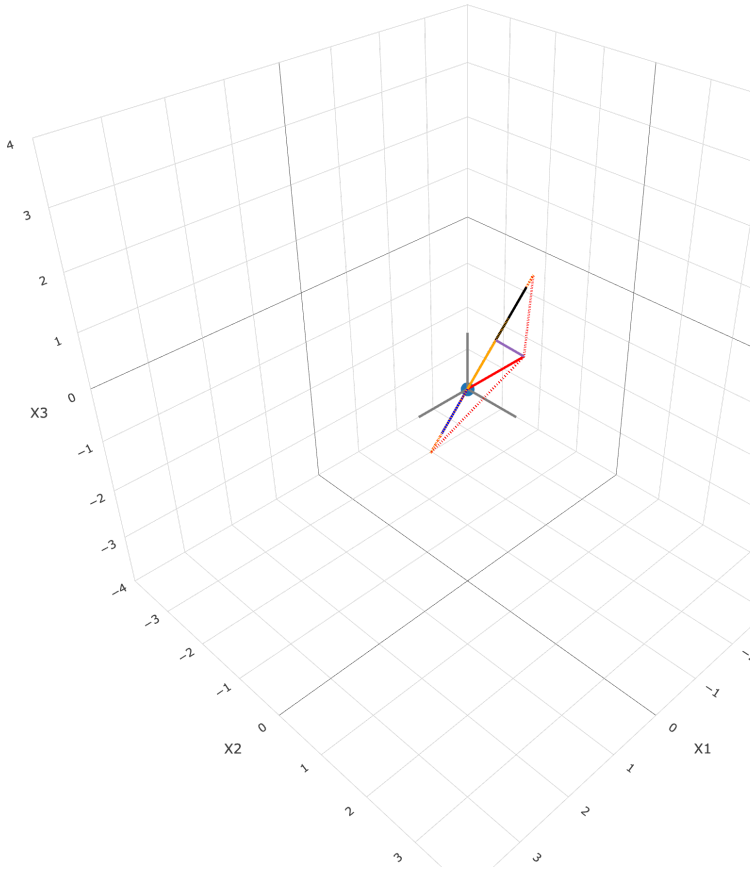
- and $\mathbf{U}_2^T\mathbf{Y}$

```
p5 <- p4 |>
  add_trace(
    x = c(0,U[1,2]*(U[,2]%*%Y)),
    y = c(0,U[2,2]*(U[,2]%*%Y)),
    z = c(0,U[3,2]*(U[,2]%*%Y)),
    mode = "lines",
    line = list(width = 5, color = "red",dash="dash"),
    type="scatter3d",
    name="Y -> U2") |> add_trace(
    x = c(Y[1],U[1,2]*(U[,2]%*%Y)),
    y = c(Y[2],U[2,2]*(U[,2]%*%Y)),
    z = c(Y[3],U[3,2]*(U[,2]%*%Y)),
    mode = "lines",
    line = list(width = 5, color = "red", dash="dash"),
    type="scatter3d",
    name="Y -> U2")
p5
```


X3
X2
X1

- Yhat is the resulting vector that lies in the plane spanned by $\mathbf{U}_1$ and $\mathbf{U}_2$ and thus also in the column space of $\mathbf{X}$.

```
p6 <- p5 |>
  add_trace(
    x = c(0,Yhat[1]),
    y = c(0,Yhat[2]),
    z = c(0,Yhat[3]),
    mode = "lines",
    line = list(width = 5, color = "orange"),
    type="scatter3d",
    name = "Yhat") |>
  add_trace(
    x = c(Y[1],Yhat[1]),
    y = c(Y[2],Yhat[2]),
    z = c(Y[3],Yhat[3]),
    mode = "lines",
    line = list(width = 5, color = "maroon2"),
    type="scatter3d",
    name = "e") |>
  add_trace(
    x = c(U[1,1]*(U[,1]%*%Y),Yhat[1]),
    y = c(U[2,1]*(U[,1]%*%Y),Yhat[2]),
    z = c(U[3,1]*(U[,1]%*%Y),Yhat[3]),
    mode = "lines",
    line = list(width = 5, color = "orange", dash="dash"),
    type="scatter3d",
    name = "Y -> U")  |>
  add_trace(
    x = c(U[1,2]*(U[,2]%*%Y),Yhat[1]),
    y = c(U[2,2]*(U[,2]%*%Y),Yhat[2]),
    z = c(U[3,2]*(U[,2]%*%Y),Yhat[3]),
    mode = "lines",
    line = list(width = 5, color = "orange", dash="dash"),
    type="scatter3d",
    name = "Y -> U")
p6
```


X3
X2
X1

3.4.3 Error

Note, that it is also clear from the equation in the derivation of the least squares solution that the residual is orthogonal on the column space:

$$-2\mathbf{X}^T(\mathbf{Y} - \mathbf{X}\beta) = 0$$

3.4.4 Curse of dimensionality?

- Imagine what happens when p approaches n $p = n$ or becomes much larger than p » n!!!
- Suppose that we add a predictor $\mathbf{X}_2 = [2, 0, 1]^T$?

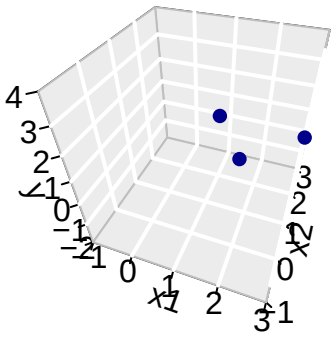
$$\mathbf{Y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 0 \\ 1 & 3 & 1 \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix} \quad \text{and} \quad \epsilon = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \end{bmatrix}$$

```
data$x2 <- c(2,0,1)
fit <- lm(y~x+x2,data)
# predict values on regular xy grid
x1pred <- seq(-1, 4, length.out = 10)
x2pred <- seq(-1, 4, length.out = 10)
xy <- expand.grid(x = x1pred,
x2 = x2pred)
ypred <- matrix (nrow = 30, ncol = 30,
data = predict(fit, newdata = data.frame(xy)))

library(plot3D)


# fitted points for droplines to surface
th=20
ph=5
scatter3D(data$x,
  data$x2,
  Y,
  pch = 16,
  col="darkblue",
  cex = 1,
  theta = th,
  ticktype = "detailed",
```

```
  xlab = "x1",
  ylab = "x2",
  zlab = "y",
  colvar=FALSE,
  bty = "g",
  xlim=c(-1,3),
  ylim=c(-1,3),
  zlim=c(-2,4))
```



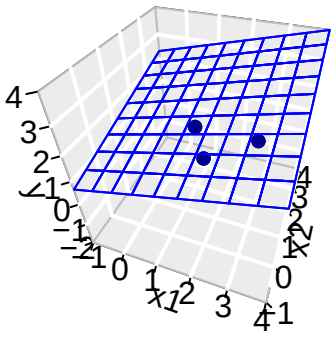

```
z.pred3D <- outer(
  x1pred,
  x2pred,
  function(x1,x2)
  {
    fit$coef[1] + fit$coef[2]*x1+fit$coef[2]*x2
  })

x.pred3D <- outer(
  x1pred,
  x2pred,
  function(x,y) x)
```

```
y.pred3D <- outer(
  x1pred,
  x2pred,
  function(x,y) y)

scatter3D(data$x,
  data$x2,
  data$y,
  pch = 16,
  col="darkblue",
  cex = 1,
  theta = th,
  ticktype = "detailed",
  xlab = "x1",
  ylab = "x2",
  zlab = "y",
  colvar=FALSE,
  bty = "g",
  xlim=c(-1,4),
  ylim=c(-1,4),
  zlim=c(-2,4))

surf3D(
  x.pred3D,
  y.pred3D,
  z.pred3D,
  col="blue",
  facets=NA,
  add=TRUE)
```



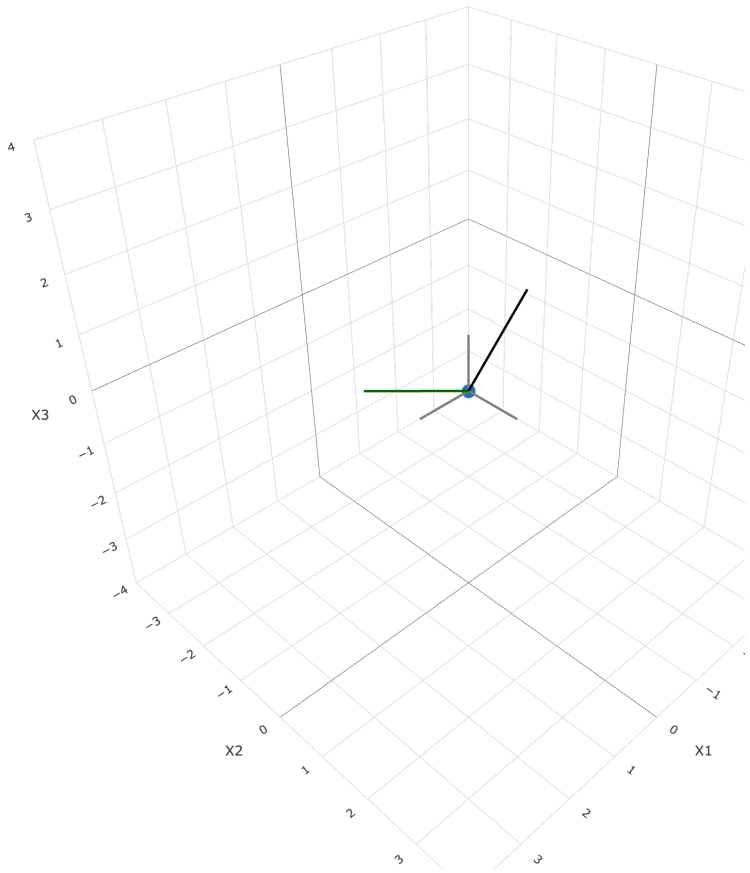
Note, that the linear regression is now a plane.

However, we obtain a perfect fit and all the data points are falling in the plane!

This is obvious if we look at the column space of X!

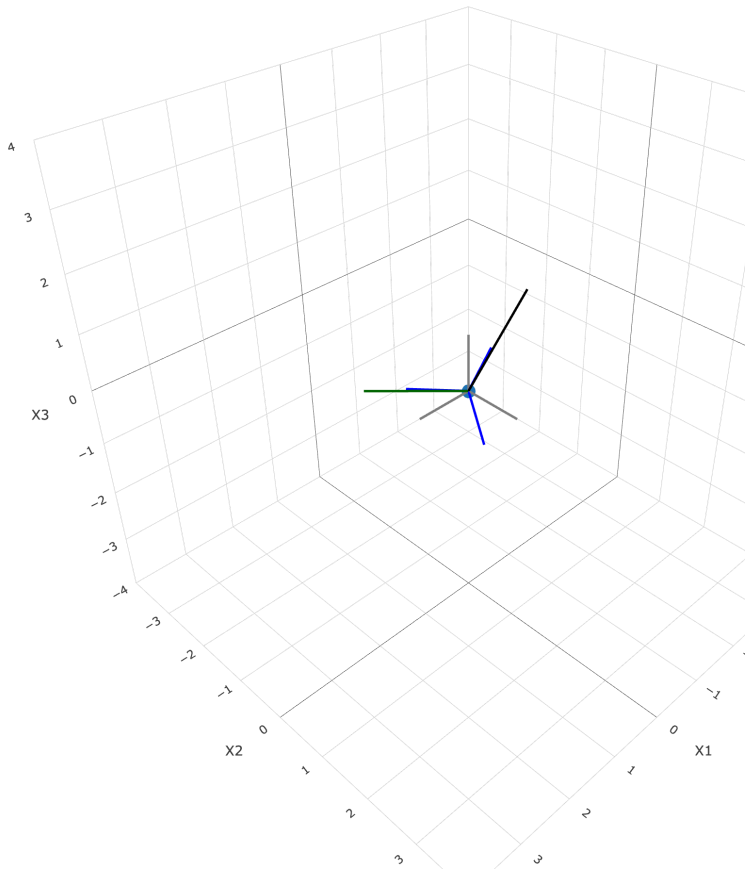
```
X <- cbind(X,c(2,0,1))
XtX <- t(X)%*%X
eigenXtX <- eigen(XtX)
XtXinvSqrt <- eigenXtX$vectors %*%diag(1/eigenXtX$values^.5)%*%t(eigenXtX$vectors)
U <- X %*% XtXinvSqrt

p7 <- p1 |>
  add_trace(
    x = c(0,2),
    y = c(0,0),
    z = c(0,1),
    mode = "lines",
    line = list(width = 5, color = "darkgreen"),
    type="scatter3d",
    name = "X3")
p7
```


4
3
2
1
0
−1
−2
−3
−4
X3
−3
−2
−1
0
1
2
3
X2
−1
0
1
2
3
X1

```r
p8 <- p7 |>
  add_trace(
    x = c(0,U[1,1]),
    y = c(0,U[2,1]),
    z = c(0,U[3,1]),
    mode = "lines",
    line = list(width = 5, color = "blue"),
    type="scatter3d",
    name = "U1") |>
  add_trace(
    x = c(0,U[1,2]),
    y = c(0,U[2,2]),
    z = c(0,U[3,2]),
    mode = "lines",
    line = list(width = 5, color = "blue"),
    type="scatter3d",
    name = "U2") |>
  add_trace(
    x = c(0,U[1,3]),
    y = c(0,U[2,3]),
    z = c(0,U[3,3]),
    mode = "lines",
    line = list(width = 5, color = "blue"),
    type="scatter3d",
    name = "U3")

p8
```


X3
X2
X1

- The column space now spans the entire $\mathbb{R}^3$!
- With the intercept and the two predictors we can thus fit every dataset that only has 3 observations for the predictors and the response.
- So the model can no longer be used to generalise the patterns seen in the data towards the population (new observations).
- Problem of overfitting!!!
- If $p >> n$ then the problem gets even worse! Then there is even no longer a unique solution to the least squares problem…
- Indeed, then we have more vectors/dimensions/columns in X than datapoints!

3.5 Variance Estimator?

$$\begin{aligned}
\hat{\Sigma}_{\hat{\beta}} &= \text{var}\left[(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}\right] \\
&= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\text{var}\left[\mathbf{Y}\right]\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}(*) \\
&= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T(\mathbf{I}\sigma^2)\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1} \\
&= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{I} \quad \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\sigma^2 \\
&= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\sigma^2 \\
&= (\mathbf{X}^T\mathbf{X})^{-1}\sigma^2
\end{aligned}$$

(*) Under assumption that all observations $\mathbf{Y}$ are independent and identically distributed.

The fact that $\hat{\beta}$ is unbiased and has a variance of $(\mathbf{X}^T\mathbf{X})^{-1}\sigma^2$ will be important when assessing association!

3.6 Prediction error

Least squares estimators are unbiased and consistent, but these properties are not very important for prediction models.

A prediction model is considered good if it can predict well outcomes.

The **prediction error** for a prediction at predictor $\mathbf{x}$ is given by

$$Y^* - \hat{Y}(\mathbf{x}),$$

where

- $\hat{Y}(\mathbf{x}) = \mathbf{x}^T\hat{\beta}$ is the prediction at $\mathbf{x}$
- Y^* is an outcome at predictor $\mathbf{x}$

Since prediction is typically used to predict an outcome before it is observed, the outcome Y^* is not observed yet. Hence, the prediction error cannot be computed.

The problem of unobservable prediction errors is partly solved by the **expected conditional test error** (sometimes referred to as the mean squared error, MSE)

$$\text{Err}(\mathbf{x}) = E\left[\left\{Y^* - \hat{Y}(\mathbf{x})\right\}^2\right].$$

With (suppressing the dependence on $\mathbf{x}$)

$$\mu = E[\hat{Y}] \text{ and } \mu^* = E[Y^*]$$

the error can be expressed as

$$\begin{aligned}
\text{Err} &= E\left[\left\{(Y^* - \mu^*) - (\hat{Y} - \mu) + (\mu^* - \mu)\right\}^2\right] \\
&= E[(Y^* - \mu^*)^2] + E[(\hat{Y} - \mu)^2] + E[(\mu^* - \mu)^2] \\
&= \text{var}[Y^*] + \text{var}[\hat{Y}] + \text{bias}^2
\end{aligned}$$

with

$$\text{bias} = \mu - \mu^* = E[\hat{Y}] - E[Y^*],$$

the difference between the average prediction and the average outcome at $\mathbf{x}$.

The term $\text{var}[Y^*]$ (irreducible error) does not depend on the model and may therefore be ignored when Err is used for comparing prediction models.

In this introductory chapter we only aim to give a rough discussion on prediction errors. Later definitions will be refined and the notation will be more accurate. Also a more detailed discussion on the bias-variance trade-off will follow. For the moment it is sufficient to vaguely know that:

- the expected conditional test error is introduced to circumvent the problem that the prediction error cannot be observed. In later chapters we will look at estimators of the expected error.
- the expected conditional test error is in some literature also known as the *mean squared error* (MSE), but we do not adopt this terminology because MSE is also commonly used to refer to SSE divided by the residual degrees of freedom in a linear regression model.
- The identity $\text{Err} = \text{var}[\hat{Y}] + \text{var}[Y^*] + \text{bias}^2$ is known as the bias-variance trade-off. It shows that a good prediction model (i.e. a model resulting in a small Err), can be obtained by a model that shows a small bias as long as this bias is compensated with a large reduction of the variance of the predictions. A more detailed discussion will follow in later chapters.
- For prediction models with a large number of predictors we will therefore introduce penalized regression. This will induce some bias in the estimation, but will allow us to reduce the variance considerably.

Session info

Session info

```
#> [1] "2026-09-24 13:40:33 CEST"
```

```
#> - Session info ---------------------------------------------------------------
#>  setting  value
#>  version  R version 4.6.1 (2026-06-24)
#>  os       macOS Tahoe 26.1
#>  system   aarch64, darwin23
#>  ui       X11
#>  language (EN)
#>  collate  nl_BE.UTF-8
#>  ctype    nl_BE.UTF-8
#>  tz       Europe/Brussels
#>  date     2026-09-24
#>  pandoc   3.10.2 @ /opt/homebrew/bin/ (via rmarkdown)
#>  quarto   1.9.37 @ /usr/local/bin/quarto
#>
#> - Packages -------------------------------------------------------------------
#>  package              * version    date (UTC) lib source
#>  abind                  1.4-8      2024-09-12 [1] CRAN (R 4.6.0)
#>  AnnotationDbi          1.74.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
```

```
#>  AnnotationHub       * 4.2.1      2026-06-15 [1] https://bioc-release.r-universe.dev (R 4
#>  assertthat            0.2.1      2019-03-21 [1] CRAN (R 4.6.0)
#>  beachmat              2.28.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  beeswarm              0.4.0      2021-06-01 [1] CRAN (R 4.6.0)
#>  Biobase             * 2.72.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  BiocFileCache       * 3.2.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  BiocGenerics        * 0.58.1     2026-05-14 [1] https://bioc-release.r-universe.dev (R 4
#>  BiocManager           1.30.27    2025-11-14 [1] CRAN (R 4.6.0)
#>  BiocNeighbors         2.6.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  BiocParallel          1.46.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  BiocSingular          1.28.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  BiocVersion           3.23.1     2025-10-30 [1] https://bioc.r-universe.dev (R 4.6.0)
#>  Biostrings            2.80.1     2026-05-22 [1] https://bioc-release.r-universe.dev (R 4
#>  bit                   4.6.0      2025-03-06 [1] CRAN (R 4.6.0)
#>  bit64                 4.8.2      2026-05-19 [1] CRAN (R 4.6.0)
#>  blob                  1.3.0      2026-01-14 [1] CRAN (R 4.6.0)
#>  cachem                1.1.0      2024-05-16 [1] CRAN (R 4.6.0)
#>  chromote              0.5.1      2025-04-24 [1] CRAN (R 4.6.0)
#>  cli                   3.6.6      2026-04-09 [1] CRAN (R 4.6.0)
#>  codetools             0.2-20     2024-03-31 [1] CRAN (R 4.6.1)
#>  cowplot               1.2.0      2025-07-07 [1] CRAN (R 4.6.0)
#>  crayon                1.5.3      2024-06-20 [1] CRAN (R 4.6.0)
#>  crosstalk             1.2.2      2025-08-26 [1] CRAN (R 4.6.0)
#>  curl                  7.1.0      2026-04-22 [1] CRAN (R 4.6.0)
#>  data.table            1.18.4     2026-05-06 [1] CRAN (R 4.6.0)
#>  DBI                   1.3.0      2026-02-25 [1] CRAN (R 4.6.0)
#>  dbplyr              * 2.6.0      2026-06-17 [1] CRAN (R 4.6.0)
#>  DelayedArray          0.38.2     2026-05-26 [1] https://bioc-release.r-universe.dev (R 4
#>  digest                0.6.39     2025-11-19 [1] CRAN (R 4.6.0)
#>  dplyr               * 1.2.1      2026-04-03 [1] CRAN (R 4.6.0)
#>  emo                   0.0.0.9000 2026-09-15 [1] Github (hadley/emo@3f03b11)
#>  evaluate              1.0.5      2025-08-27 [1] CRAN (R 4.6.0)
#>  ExperimentHub       * 3.2.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  farver                2.1.2      2024-05-13 [1] CRAN (R 4.6.0)
#>  fastmap               1.2.0      2024-05-15 [1] CRAN (R 4.6.0)
#>  filelock              1.0.3      2023-12-11 [1] CRAN (R 4.6.0)
#>  forcats             * 1.0.1      2025-09-25 [1] CRAN (R 4.6.0)
#>  generics            * 0.1.4      2025-05-09 [1] CRAN (R 4.6.0)
#>  GenomicRanges       * 1.64.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  ggbeeswarm            0.7.3      2025-11-29 [1] CRAN (R 4.6.0)
#>  ggplot2             * 4.0.3      2026-04-22 [1] CRAN (R 4.6.0)
#>  ggrepel               0.9.8      2026-03-17 [1] CRAN (R 4.6.0)
#>  glue                  1.8.1      2026-04-17 [1] CRAN (R 4.6.0)
```

```
#>  gridExtra             2.3.1      2026-06-25 [1] CRAN (R 4.6.1)
#>  gtable                0.3.6      2024-10-25 [1] CRAN (R 4.6.0)
#>  hms                   1.1.4      2025-10-17 [1] CRAN (R 4.6.0)
#>  htmltools             0.5.9      2025-12-04 [1] CRAN (R 4.6.0)
#>  htmlwidgets           1.6.4      2023-12-06 [1] CRAN (R 4.6.0)
#>  httr                  1.4.8      2026-02-13 [1] CRAN (R 4.6.0)
#>  httr2                 1.2.3      2026-06-23 [1] CRAN (R 4.6.1)
#>  IRanges             * 2.46.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  irlba                 2.3.7      2026-01-30 [1] CRAN (R 4.6.0)
#>  jsonlite              2.0.0      2025-03-27 [1] CRAN (R 4.6.0)
#>  KEGGREST              1.52.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  knitr                 1.51       2025-12-20 [1] CRAN (R 4.6.0)
#>  labeling              0.4.3      2023-08-29 [1] CRAN (R 4.6.0)
#>  later                 1.4.8      2026-03-05 [1] CRAN (R 4.6.0)
#>  lattice               0.22-9     2026-02-09 [1] CRAN (R 4.6.1)
#>  lazyeval              0.2.3      2026-04-04 [1] CRAN (R 4.6.0)
#>  lifecycle             1.0.5      2026-01-08 [1] CRAN (R 4.6.0)
#>  lubridate           * 1.9.5      2026-02-04 [1] CRAN (R 4.6.0)
#>  magrittr              2.0.5      2026-04-04 [1] CRAN (R 4.6.0)
#>  Matrix                1.7-5      2026-03-21 [1] CRAN (R 4.6.1)
#>  MatrixGenerics      * 1.24.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  matrixStats         * 1.5.0      2025-01-07 [1] CRAN (R 4.6.0)
#>  memoise               2.0.1      2021-11-26 [1] CRAN (R 4.6.0)
#>  mgcv                  1.9-4      2025-11-07 [1] CRAN (R 4.6.1)
#>  misc3d                0.9-2      2026-01-28 [1] CRAN (R 4.6.0)
#>  nlme                  3.1-169    2026-03-27 [1] CRAN (R 4.6.1)
#>  otel                  0.2.0      2025-08-29 [1] CRAN (R 4.6.0)
#>  pillar                1.11.1     2025-09-17 [1] CRAN (R 4.6.0)
#>  pkgconfig             2.0.3      2019-09-22 [1] CRAN (R 4.6.0)
#>  plot3D              * 1.4.2      2025-07-25 [1] CRAN (R 4.6.0)
#>  plotly              * 4.12.0     2026-01-24 [1] CRAN (R 4.6.0)
#>  png                   0.1-9      2026-03-15 [1] CRAN (R 4.6.0)
#>  processx              3.9.0      2026-04-22 [1] CRAN (R 4.6.0)
#>  promises              1.5.0      2025-11-01 [1] CRAN (R 4.6.0)
#>  purrr               * 1.2.2      2026-04-10 [1] CRAN (R 4.6.0)
#>  R6                    2.6.1      2025-02-15 [1] CRAN (R 4.6.0)
#>  rappdirs              0.3.4      2026-01-17 [1] CRAN (R 4.6.0)
#>  RColorBrewer          1.1-3      2022-04-03 [1] CRAN (R 4.6.0)
#>  Rcpp                  1.1.1-1.1  2026-04-24 [1] CRAN (R 4.6.0)
#>  readr               * 2.2.0      2026-02-19 [1] CRAN (R 4.6.0)
#>  rlang                 1.2.0      2026-04-06 [1] CRAN (R 4.6.0)
#>  rmarkdown             2.31       2026-03-26 [1] CRAN (R 4.6.0)
#>  RSQLite               3.53.2     2026-06-17 [1] CRAN (R 4.6.0)
```

```
#>  rsvd                    1.0.5      2021-04-16 [1] CRAN (R 4.6.0)
#>  S4Arrays                1.12.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  S4Vectors             * 0.50.1     2026-05-03 [1] https://bioc-release.r-universe.dev (R 4
#>  S7                      0.2.2      2026-04-22 [1] CRAN (R 4.6.0)
#>  ScaledMatrix            1.20.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  scales                  1.4.0      2025-04-24 [1] CRAN (R 4.6.0)
#>  scater                * 1.40.1     2026-05-20 [1] https://bioc-release.r-universe.dev (R 4
#>  scuttle               * 1.22.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  Seqinfo               * 1.2.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  sessioninfo             1.2.4      2026-06-04 [1] CRAN (R 4.6.0)
#>  SingleCellExperiment * 1.34.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  SparseArray             1.12.2     2026-05-01 [1] Bioconductor 3.23 (R 4.6.0)
#>  stringi                 1.8.7      2025-03-27 [1] CRAN (R 4.6.0)
#>  stringr               * 1.6.0      2025-11-04 [1] CRAN (R 4.6.0)
#>  SummarizedExperiment * 1.42.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  tibble                * 3.3.1      2026-01-11 [1] CRAN (R 4.6.0)
#>  tidyr                 * 1.3.2      2025-12-19 [1] CRAN (R 4.6.0)
#>  tidyselect              1.2.1      2024-03-11 [1] CRAN (R 4.6.0)
#>  tidyverse             * 2.0.0      2023-02-22 [1] CRAN (R 4.6.0)
#>  timechange              0.4.0      2026-01-29 [1] CRAN (R 4.6.0)
#>  tzdb                    0.5.0      2025-03-15 [1] CRAN (R 4.6.0)
#>  vctrs                   0.7.3      2026-04-11 [1] CRAN (R 4.6.0)
#>  vipor                   0.4.7      2023-12-18 [1] CRAN (R 4.6.0)
#>  viridis                 0.6.5      2024-01-29 [1] CRAN (R 4.6.0)
#>  viridisLite             0.4.3      2026-02-04 [1] CRAN (R 4.6.0)
#>  webshot2                0.1.2      2025-04-23 [1] CRAN (R 4.6.0)
#>  websocket               1.4.4      2025-04-10 [1] CRAN (R 4.6.0)
#>  withr                   3.0.3      2026-06-19 [1] CRAN (R 4.6.0)
#>  xfun                    0.59       2026-06-19 [1] CRAN (R 4.6.0)
#>  XVector                 0.52.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  yaml                    2.3.12     2025-12-10 [1] CRAN (R 4.6.0)
#>
#>  [1] /Library/Frameworks/R.framework/Versions/4.6/Resources/library
#>  * -- Packages attached to the search path.
#>
#> ------------------------------------------------------------------------------
```